

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

March 05, 2026
Thursday



$$\begin{aligned} & f(x) \\ & \min_x x^2 \\ & \text{s.t. } x \geq 1 \\ & \max_{\lambda} \end{aligned} \quad \left. \begin{aligned} & x^2 \\ & x \geq 1 \end{aligned} \right\} \text{primal}$$

$\rightarrow$ free no. of hours

$$\begin{aligned} x &\geq 1 \\ x-1 &\geq 0 \\ \lambda(x-1) &\geq 0 \\ &\hookrightarrow \lambda \geq 0 \end{aligned}$$

$$L = x^2 - \lambda(x-1)$$

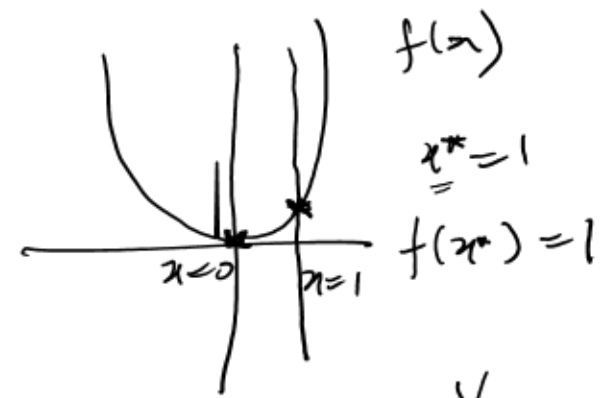
$$\min_{x, \lambda} x^2 - \lambda(x-1)$$

$$\frac{\partial L}{\partial x} = 2x - \lambda = 0$$

$$\left[x = \frac{\lambda}{2} \right]$$

$$\left(\frac{\lambda}{2} \right)^2 - \lambda \left(\frac{\lambda}{2} - 1 \right)$$

$$y(\lambda)_{\max} = \frac{\lambda^2}{4} + \lambda \quad \text{dual} = 1$$



$$\begin{aligned} x &= \lambda/2 \\ \lambda &= 2x \\ \lambda^* &= 2x^* \\ \lambda^* &= 2 \\ &= \end{aligned}$$

$$f^*(x) = g^*(x)$$

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

st

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

primal

$$\min_{w,b,\xi} \max_{\alpha,\mu} \mathcal{L} = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \underbrace{\sum_i \alpha_i [y_i(w^T x_i + b) - 1 + \xi_i]}_{\geq 0} - \underbrace{\sum_i \mu_i \xi_i}_{\geq 0}$$

Lagrange multipliers

$\alpha_i \geq 0$ for margin constraints

$\mu_i \geq 0$ for slack constraints

$$L = \frac{\partial L}{\partial w} \quad w - \sum_i \alpha_i y_i x_i = 0 \rightarrow \textcircled{1}$$

w, b, ε

$w = \sum_i \alpha_i y_i x_i$ } weighted sum of training points.

$$\frac{\partial L}{\partial b} \quad \sum_i \alpha_i y_i = 0 \quad \text{constraint} \quad \textcircled{2}$$

$$\frac{\partial L}{\partial \varepsilon} \quad c - \alpha_i - \mu_i = 0 \quad \mu_i \geq 0$$

$$c \nmid \alpha_i = \mu_i$$

$$c \nmid \alpha_i \geq 0$$

$$\alpha_i \leq c$$

$$0 \leq \alpha_i \leq c \quad \text{constraint} \quad \textcircled{3}$$

New dual SVM formulation

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{x_i^T x_j}$$

s.t

$$0 \leq \alpha_i \leq C$$

$$\sum_i \alpha_i y_i = 0$$

$\alpha_i > 0 \rightarrow$ support vectors
 $\alpha_i = 0 \rightarrow$ we don't care about training points on testing

Support vectors and alpha

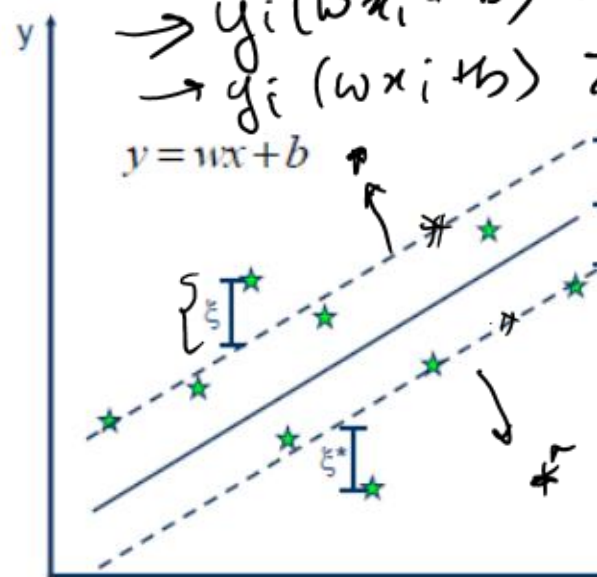
$$\mathcal{L} = \left(\frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \underbrace{\sum_i \alpha_i \underbrace{[y_i(w^T x_i + b) - 1 + \xi_i]}_{\substack{\geq 0 \\ \geq 0}}}_{\substack{\geq 0 \\ \leq 0}} - \sum_i \mu_i \xi_i \right) \quad \text{①} \quad \alpha_i \geq 0$$

$$y_i(w x_i + b) - 1 + \xi \geq 0$$

$$y_i(w x_i + b) \geq 1 - \xi$$

$$\rightarrow y_i(w x_i + b) = 1 - \xi$$

$$\rightarrow y_i(w x_i + b) \geq 1 - \xi$$



RBF Kernel

$$\begin{aligned}\exp\left(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right) &= \exp\left(\frac{2}{2}\mathbf{x}^\top \mathbf{x}' - \frac{1}{2}\|\mathbf{x}\|^2 - \frac{1}{2}\|\mathbf{x}'\|^2\right) \\&= \exp(\mathbf{x}^\top \mathbf{x}') \exp\left(-\frac{1}{2}\|\mathbf{x}\|^2\right) \exp\left(-\frac{1}{2}\|\mathbf{x}'\|^2\right) \\&= \sum_{j=0}^{\infty} \frac{(\mathbf{x}^\top \mathbf{x}')^j}{j!} \exp\left(-\frac{1}{2}\|\mathbf{x}\|^2\right) \exp\left(-\frac{1}{2}\|\mathbf{x}'\|^2\right) \\&= \sum_{j=0}^{\infty} \sum_{n_1+n_2+\dots+n_k=j} \exp\left(-\frac{1}{2}\|\mathbf{x}\|^2\right) \frac{x_1^{n_1} \dots x_k^{n_k}}{\sqrt{n_1! \dots n_k!}} \exp\left(-\frac{1}{2}\|\mathbf{x}'\|^2\right) \frac{x_1'^{n_1} \dots x_k'^{n_k}}{\sqrt{n_1! \dots n_k!}} \\&= \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle\end{aligned}$$

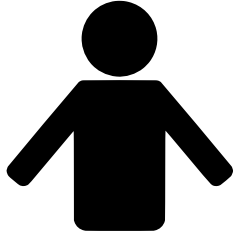
Today's goal – Learn about fully connected neural networks

- (1) Multi-layered perceptron
- (2) Quick recap of loss functions and optimizer
- (3) Building our neural network

Multi-layered Perceptron

(or Fully connected neural networks)

Recap: Heart disease classification



Input: $\mathbb{X}$

binary classification

Target: $\mathbb{Y}$



age,cholesterol,resting_bp,max_hr

45,180,120,170

48,185,118,172

50,190,125,165

52,200,130,160

55,210,135,158

57,220,140,150

58,235,142,152

60,240,145,148

62,250,150,140

65,260,155,138

What is our prediction task?



Function: f



$f(\mathbb{X}) \rightarrow \mathbb{Y}$

heart_disease

0

0

0

0

0

1

1

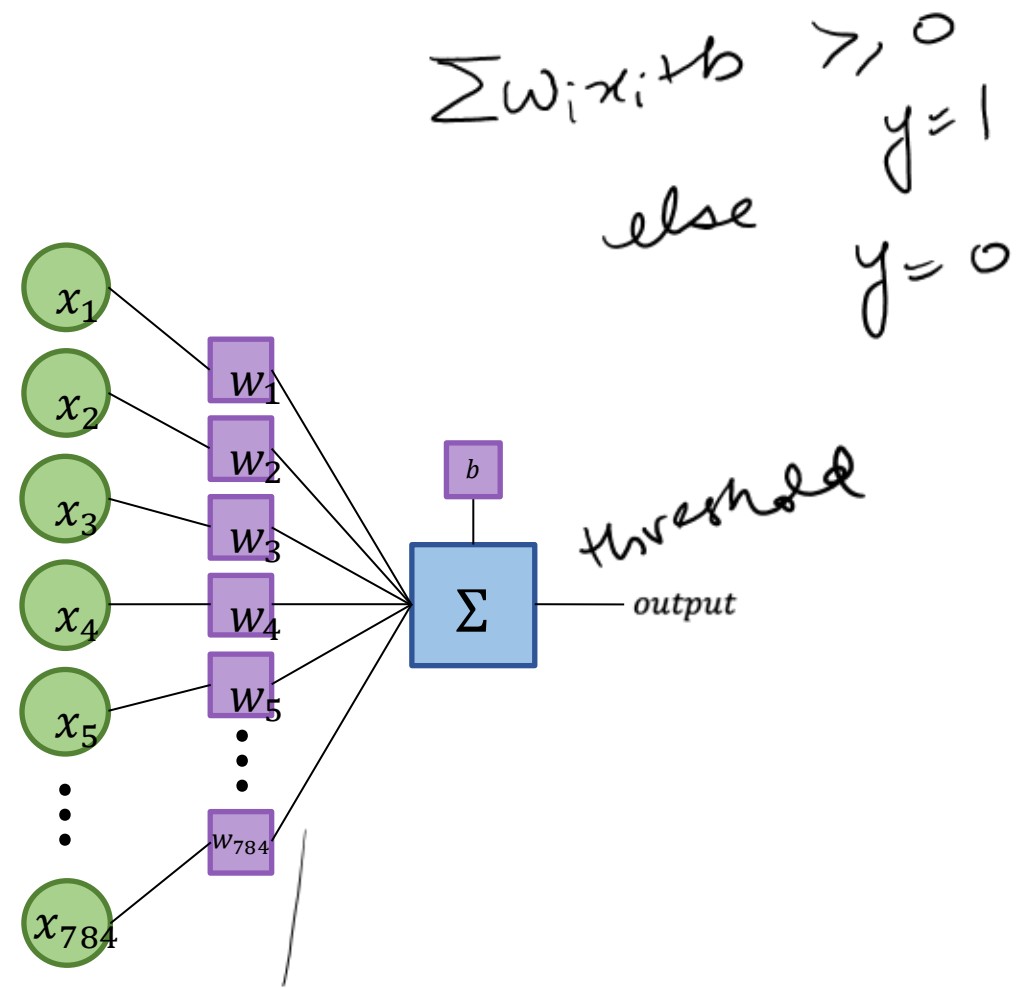
1

1

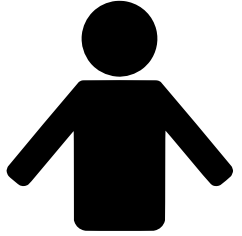
1

Recap: A Linear Perceptron for classifying heart disease

- *Inputs* $[x_1, x_2, \dots, x_n]$ are all positive
 - $n = 4$ (features)
- *output* is either 0 or 1
 - $0 \rightarrow$ no heart disease
 - $1 \rightarrow$ has heart disease



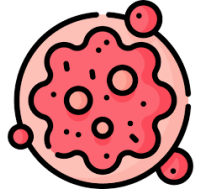
Recap: Cancer-type prediction



Input: $\mathbb{X}$

*multi-class
classification*

Target: $\mathbb{Y}$



Gene_1	Gene_2	Gene_3	Gene_4
3.24	1.95	3.90	4.38
0.90	0.01	-0.53	-1.50
4.04	2.72	6.34	3.76
2.99	2.31	3.99	3.34
1.41	1.39	-0.88	0.08
0.88	-0.24	1.21	0.54
3.82	4.19	4.39	4.00
3.05	1.53	1.33	3.01
4.14	4.28	5.86	4.97
4.51	3.81	4.23	4.90

What is our prediction task?



Function: f



$f(\mathbb{X}) \rightarrow \mathbb{Y}$

Could you use perceptrons?

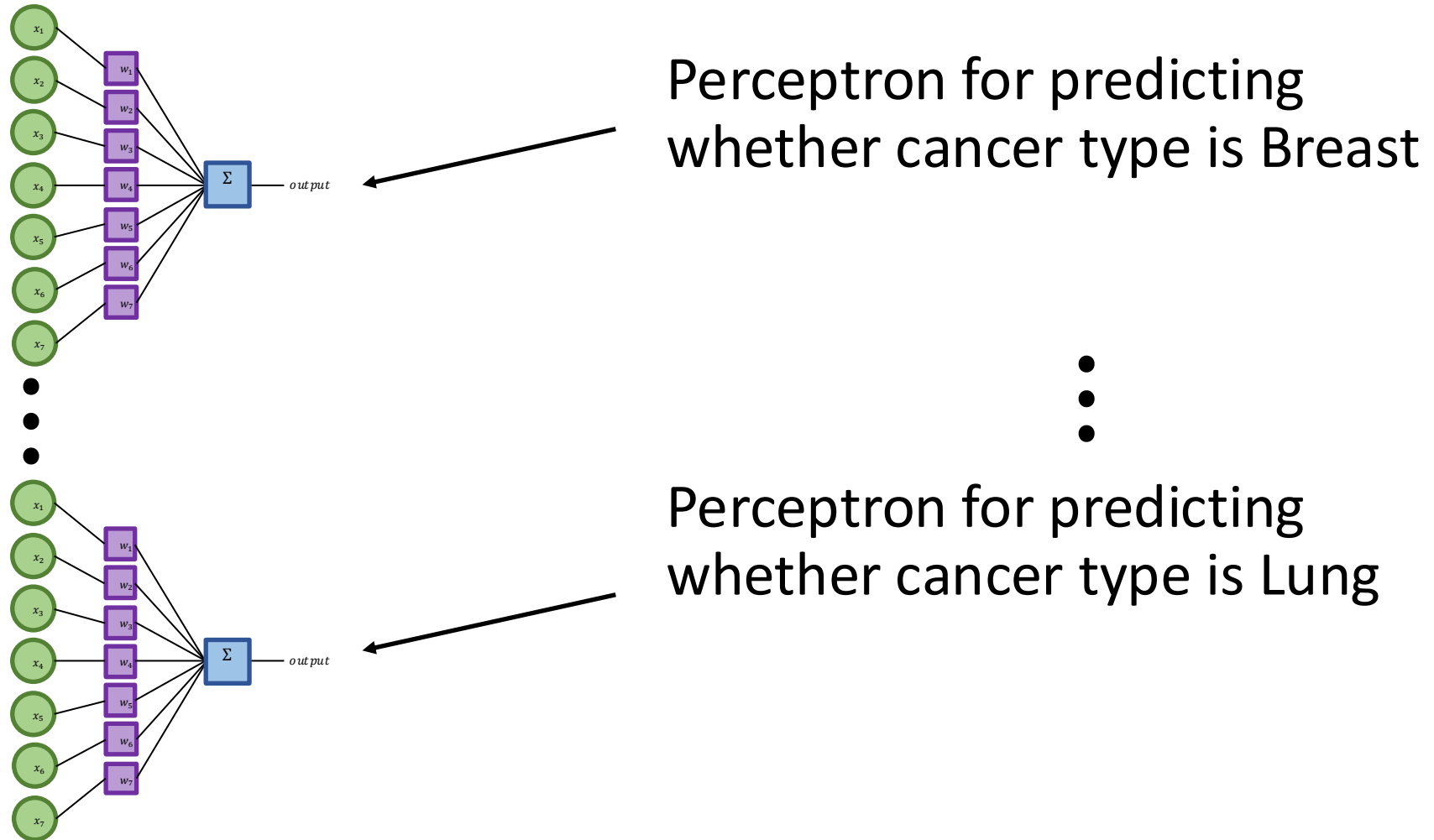
Cancer_Type

Prostate
Breast
Prostate
Prostate
Breast
Breast
Prostate
Lung
Prostate
Prostate

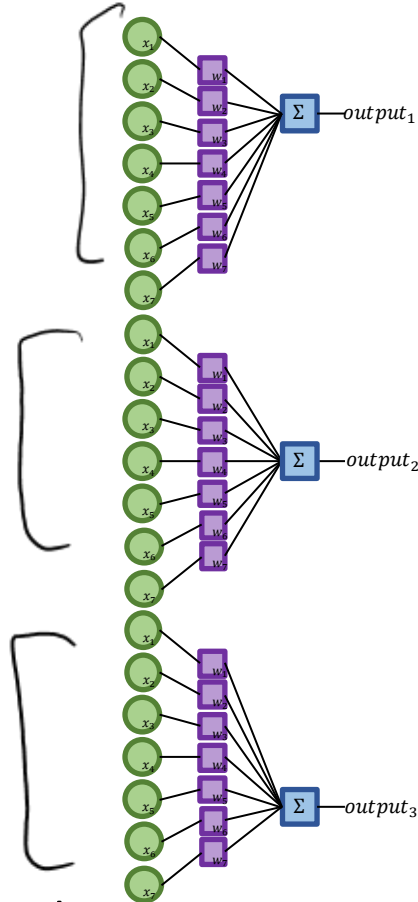
Using multiple perceptrons

- We can extend perceptrons to multi-class problems by creating m perceptrons, where m = the number of classes
- For cancer prediction, we would have 4 perceptrons for 4 classes
- Each individual perceptron returns a value, so our model will return the class whose perceptron value is the highest.
 - Here, “perceptron value” refers to the value of the weighted sum before being thresholded.

Using multiple perceptrons

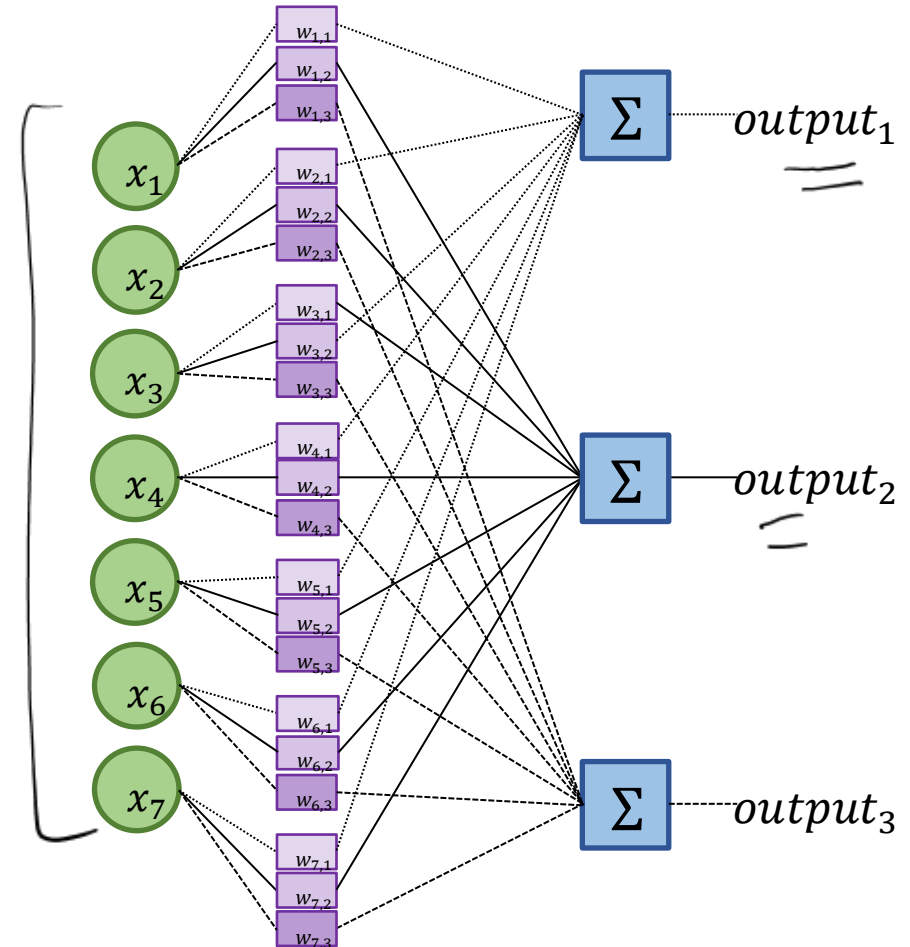


Multi-class Perceptron



Three separate perceptrons

=



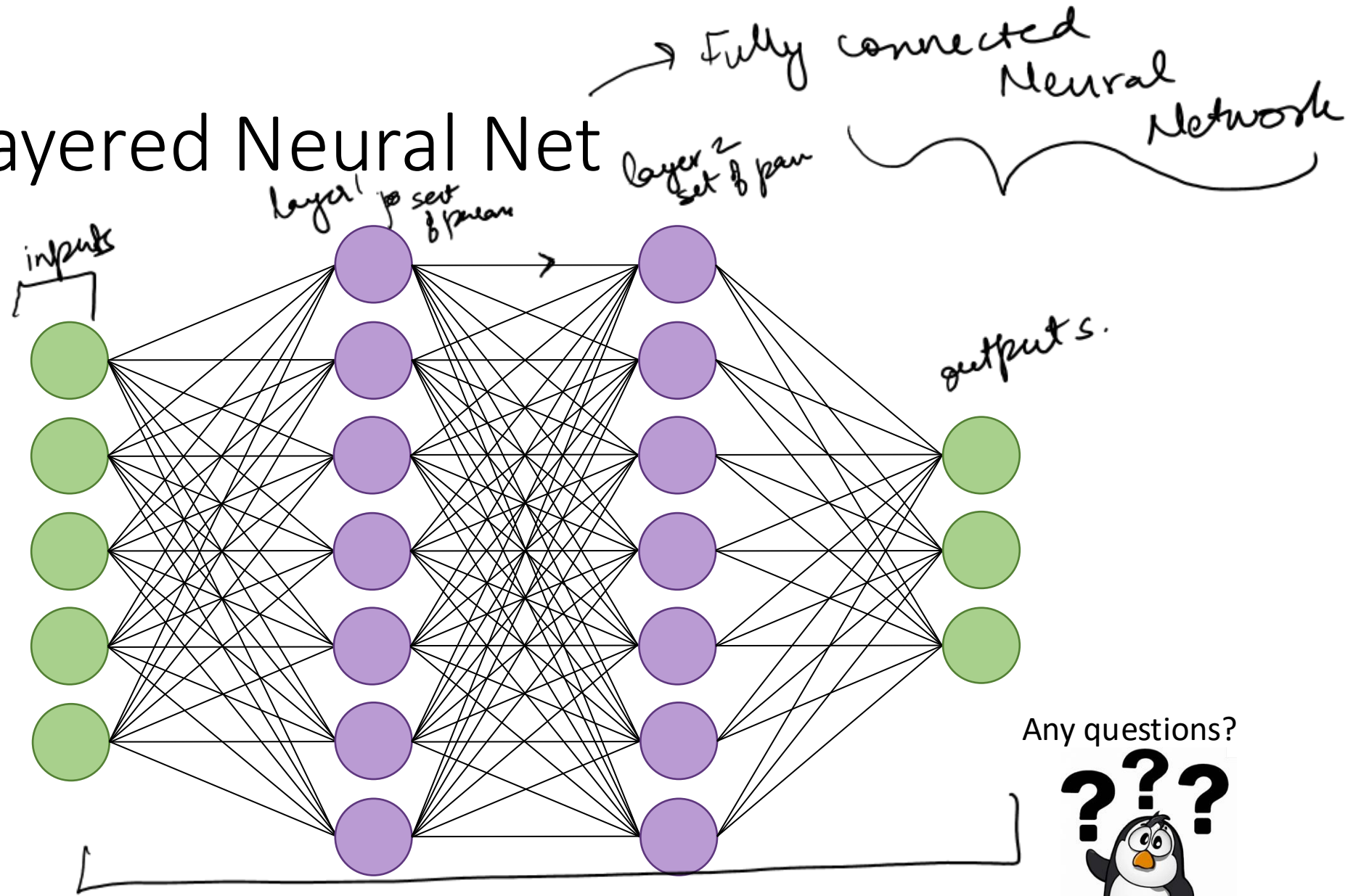
Three perceptrons sharing inputs

Is there anything the Perceptron can't learn?

→ Multi-Layered Neural Net

May see the term
Multi-Layer
Perceptron (MLP),
HOWEVER
"perceptrons" are
not perceptrons in
the strictest possible
sense

We really don't use
the threshold
function of a
perceptron but still
use the linear
function

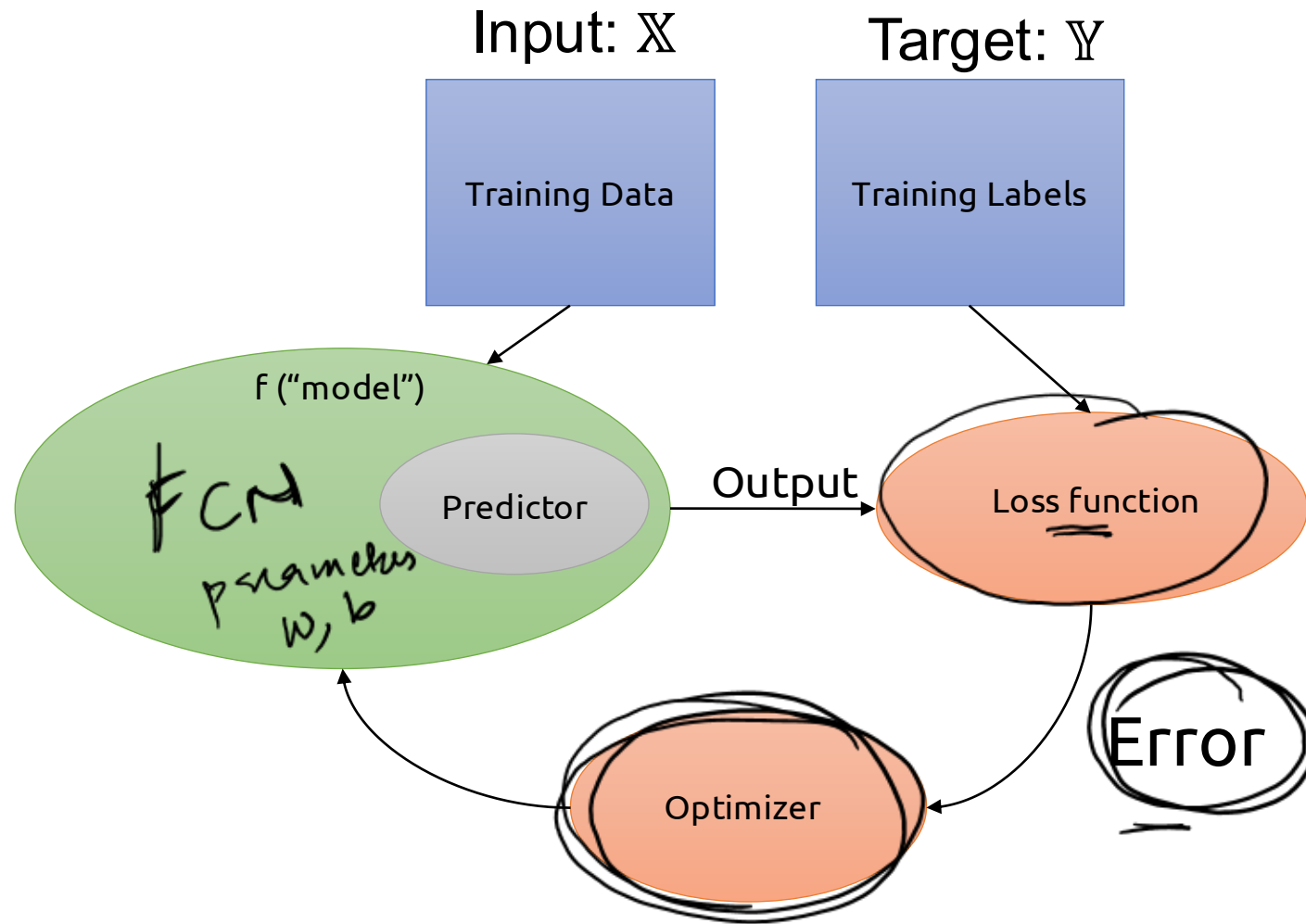


A Multi-Layered Neural Net

Any questions?



“Classic” Supervised Learning in Machine Learning



Recap of loss functions and optimizer

Recap: Mean Squared Error (MSE)

Average squared residual (residual: difference between predicted and true value)

Decreasing the MSE = the model has less error = data points fall closer to the regression line

$$MSE = \frac{\sum_{k=1}^n (y^k - \hat{y}^k)^2}{n}$$

y^k : true output value

$\hat{y}^k$: predicted output value

n : number of samples

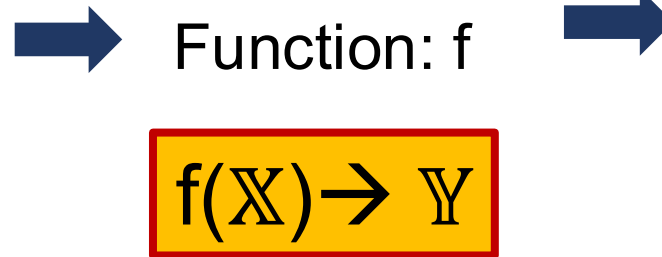
What regression task
have we seen in the
course?

Recap: Age prediction from DNA methylation

Input: $\mathbb{X}$

Target: $\mathbb{Y}$

Sample	CpG ₁	CpG ₂	CpG ₃
1	0.20	0.35	0.60
2	0.30	0.45	0.65
3	0.40	0.55	0.70
4	0.50	0.65	0.75



Age
25
35
45
55

Recap: Cross Entropy Loss (for Binary classification)

$y = \text{true label of class (0 or 1)}$

$p = \text{predicted probability of class 1}$



$$-(\underbrace{y \log(p)} + (1 - y) \log(1 - p))$$

We get the
probability by using
a Sigmoid function

Recap: Cross Entropy Loss (for Multi-class classification)

$$-\sum_{j=1}^m y_j \log(p_j)$$

↑ ↑

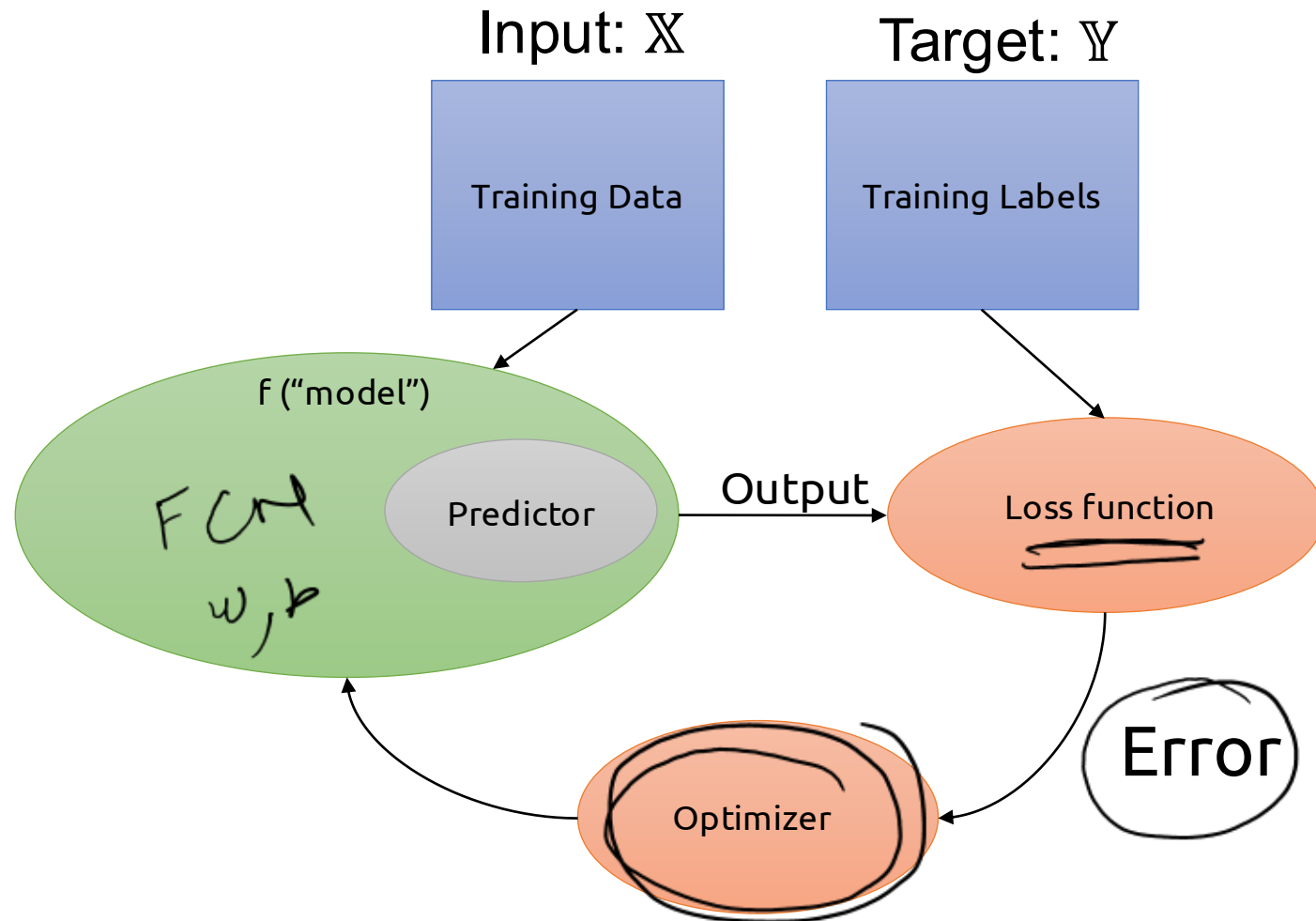
p	Classes (m)	y
0.3	"Lung"	0
0.2	"Breast"	0
0.5	"Prostrate"	1

We can get these probabilities by using a Softmax function

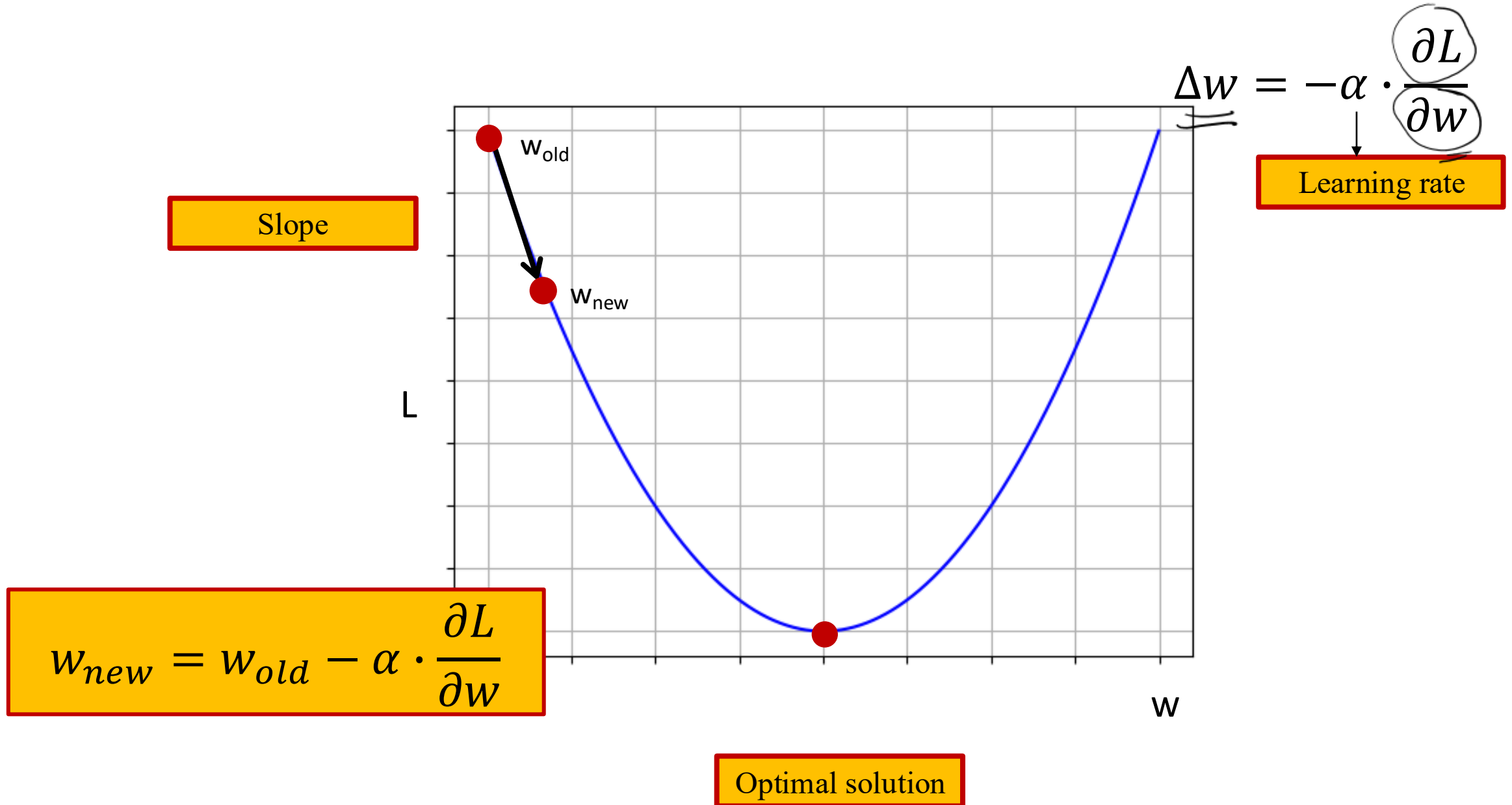
Any questions?



“Classic” Supervised Learning in Machine Learning



Recap: Gradient Descent (updating parameters)



Recap: Convex functions

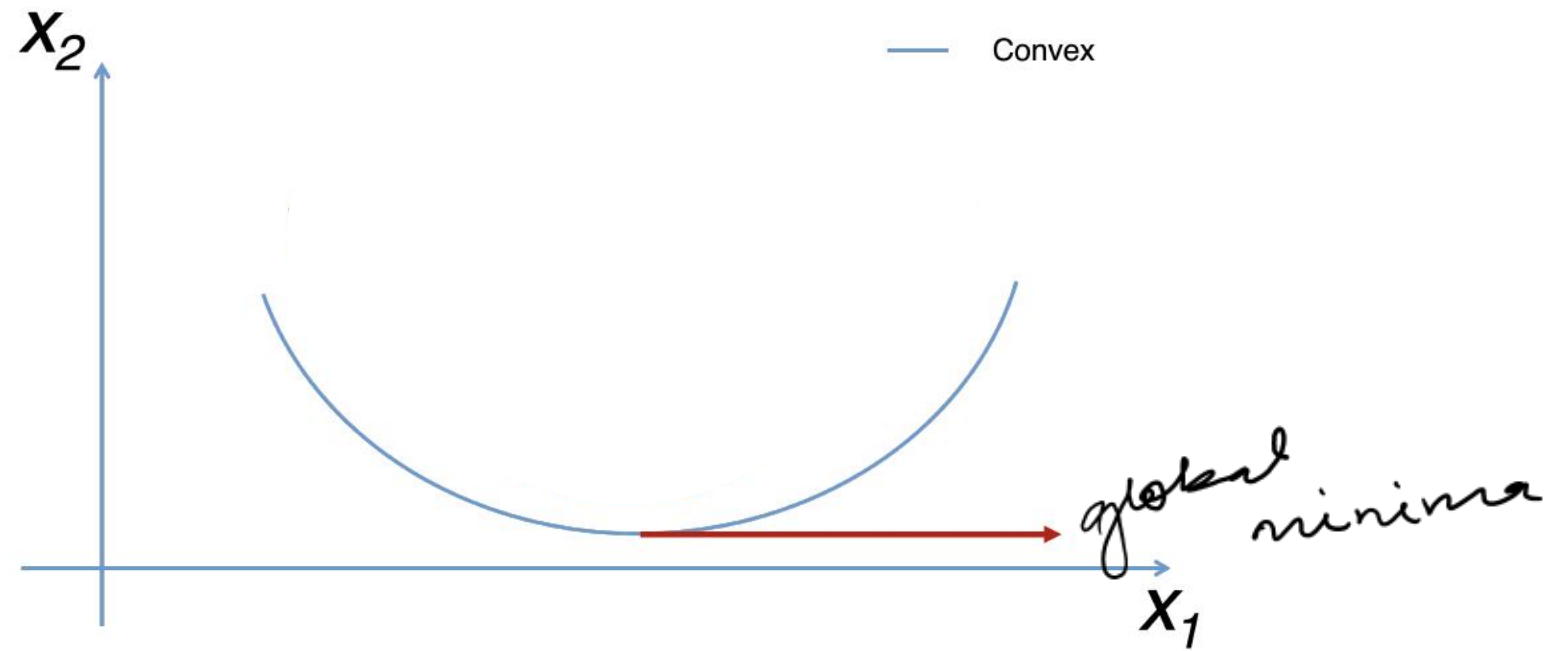


Figure: https://fmin.xyz/docs/theory/Convex_function/

Convex and Non convex functions

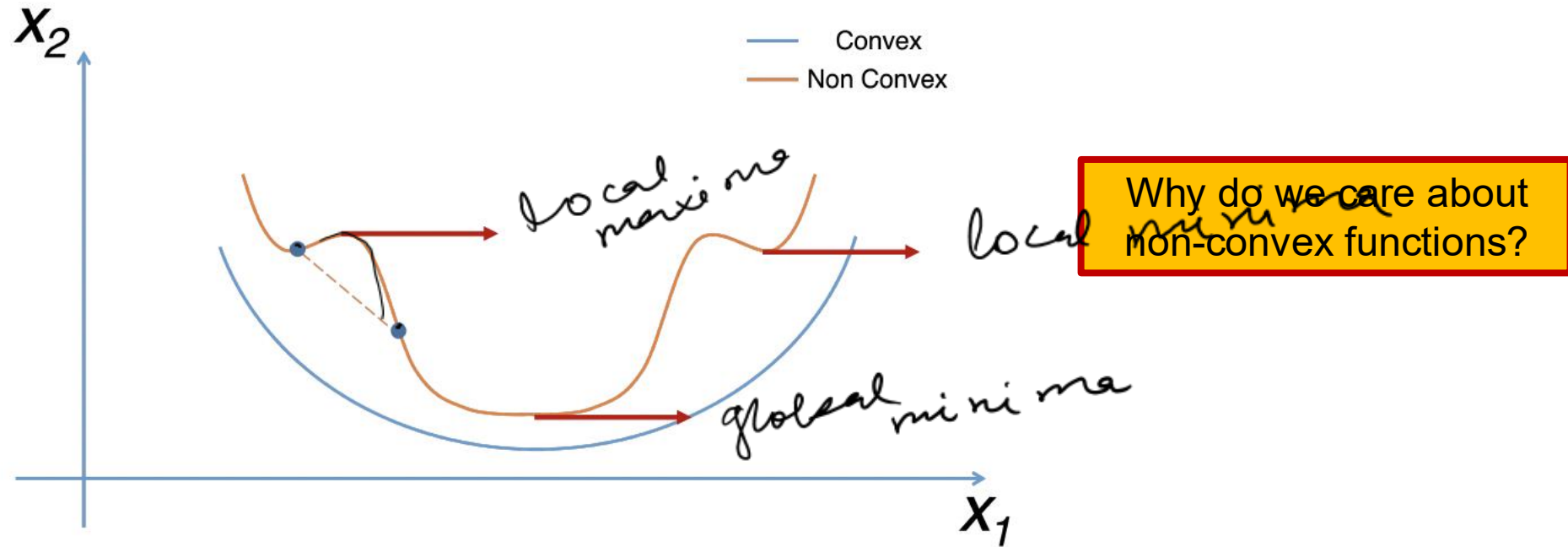
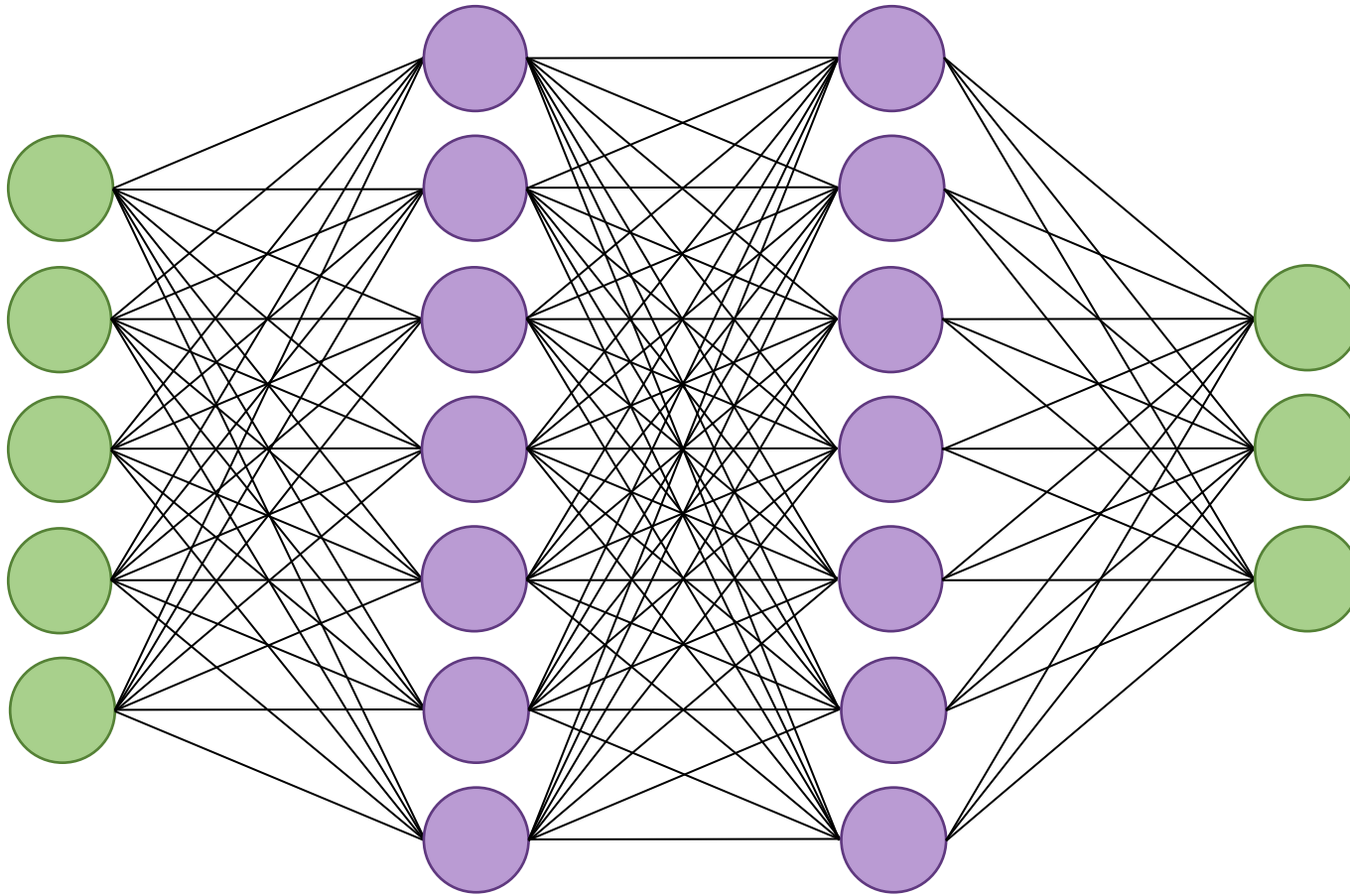
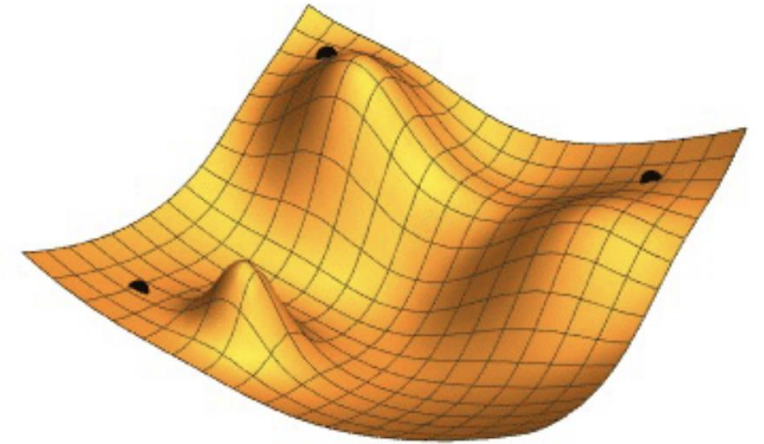


Figure: https://fmin.xyz/docs/theory/Convex_function/

Why we care about non-convex functions?



A Multi-Layered Neural Net

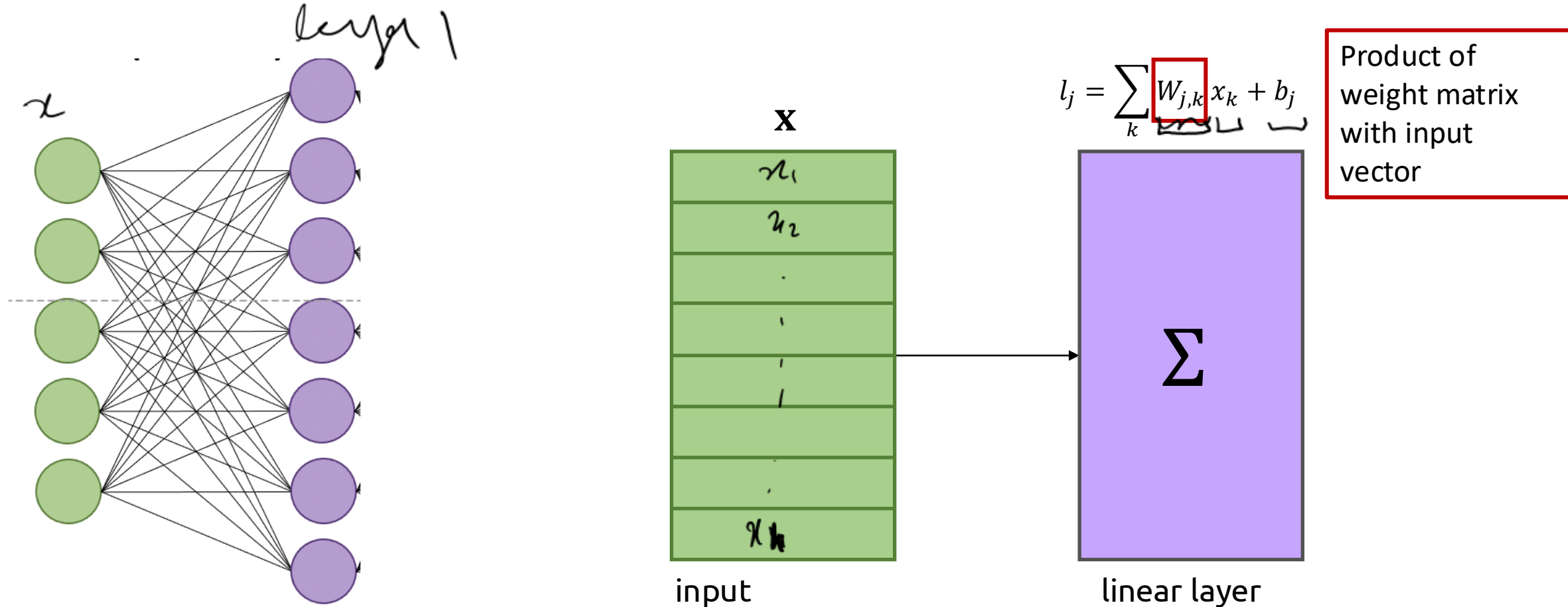


Gradient descent can
help the neural net learn!

Building neural network

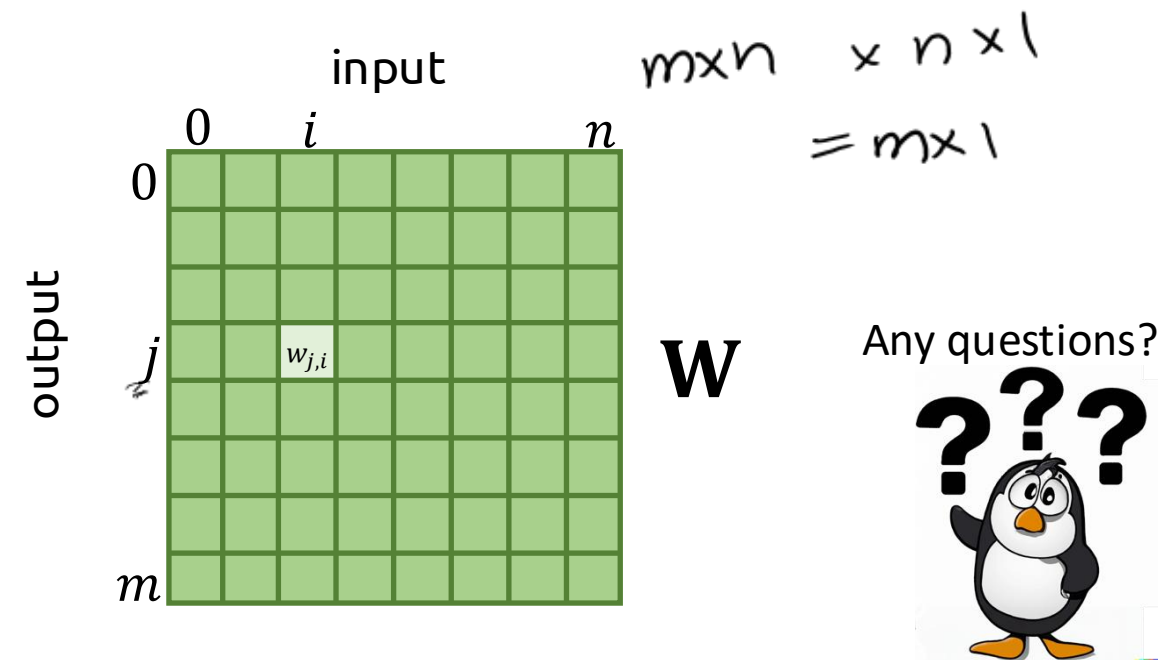
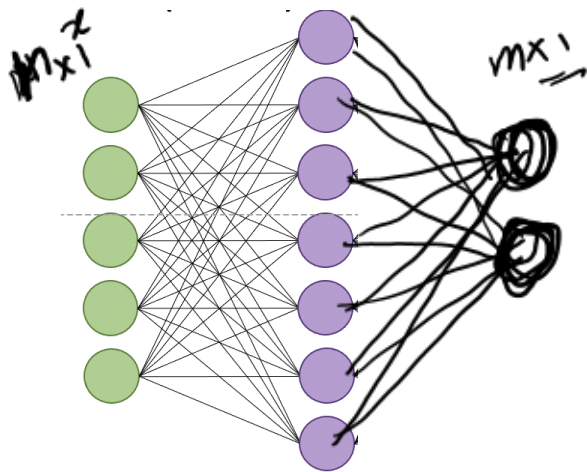
Let's start building our neural network model

- This is a simplified view of our model with an input and a linear layer

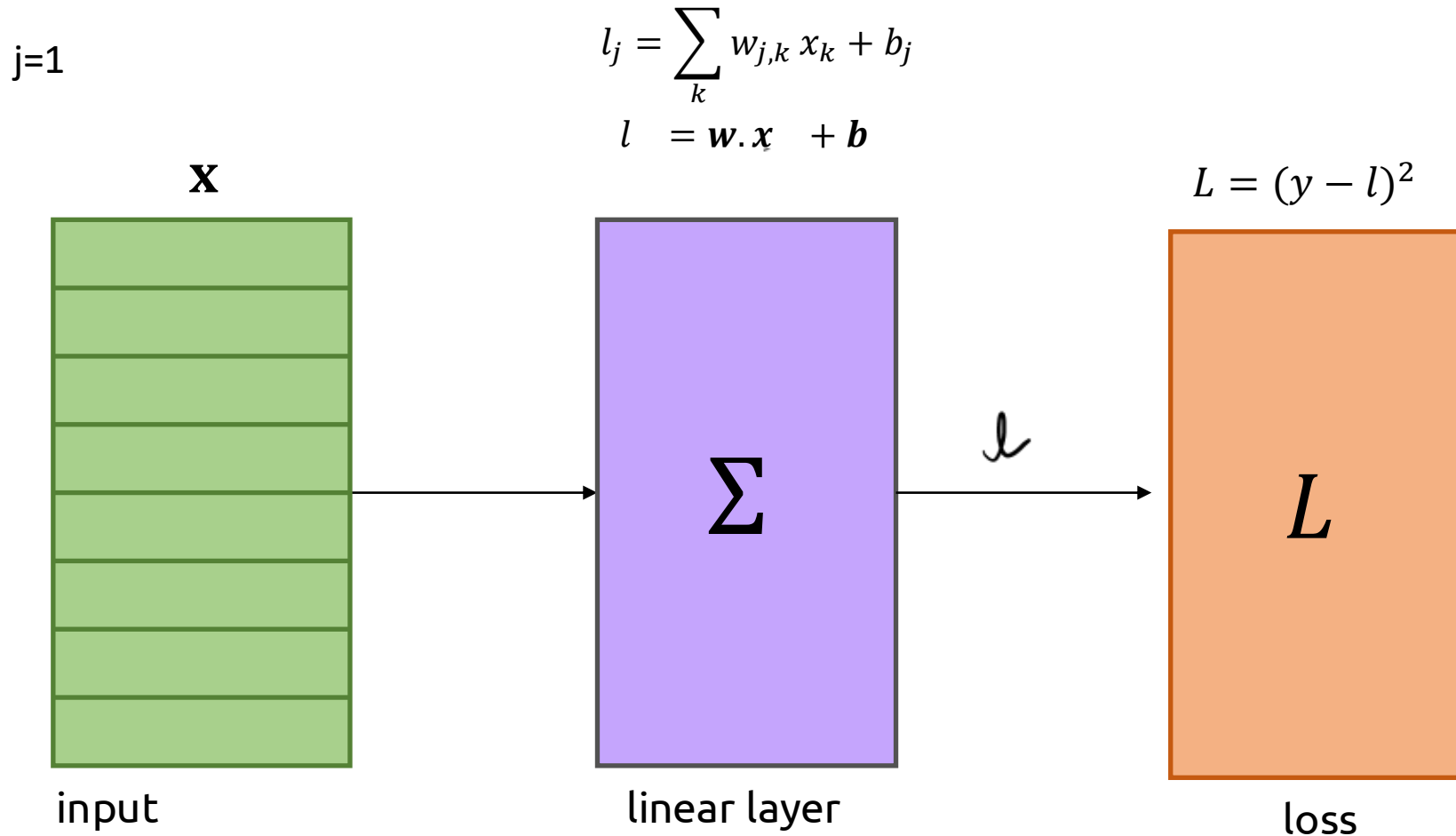


Our Weight Matrix

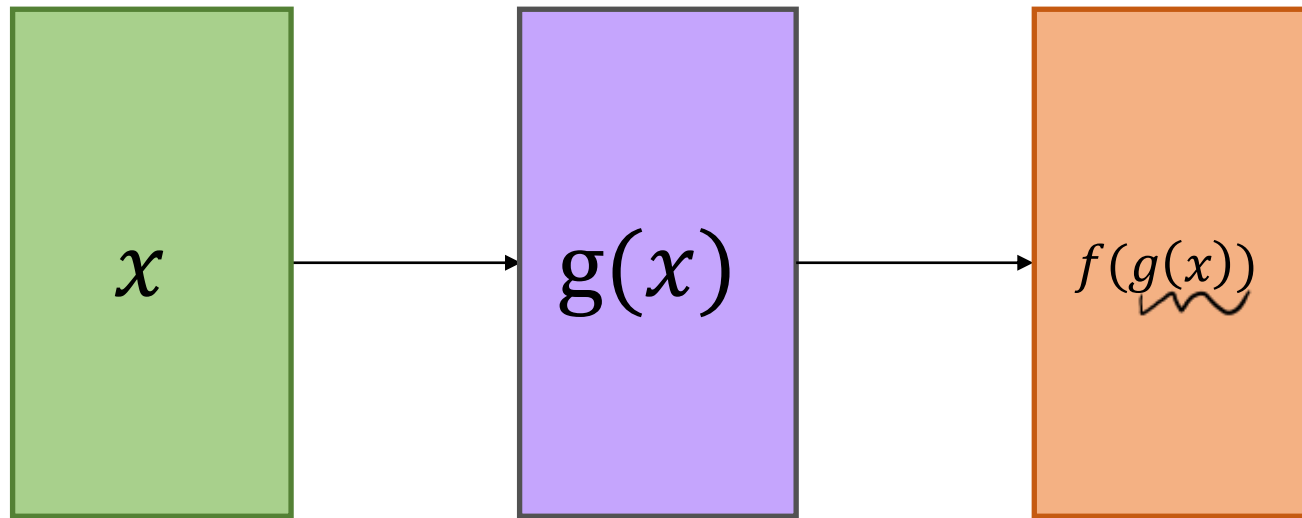
- We have an input vector of size n and an output vector of size m , so our weights matrix $\mathbf{W}$ is of dimensionality $m \times n$
- $w_{j,i}$ is the j^{th} row and the i^{th} column of our matrix, or the weight multiplied by the i^{th} index of the input which is used to create the j^{th} index in the output



Adding MSE Loss to Our Network



Looking at composite function!



Using gradient descent to update parameters

- Recall the parameter update for Gradient Descent: $\Delta w = -\alpha \cdot \frac{\partial L}{\partial w}$
- L is a composition of a series of functions (linear layers, loss layer, maybe more...)
- How do we compute the derivative of a composition of functions?
 - Hint: think back to your calculus classes...

Chain rule

If f and g are both differentiable and $F(x)$ is the composite function defined by $F(x) = f(g(x))$ then F is differentiable and F' is given by the product

$$F'(x) = f'(g(x)) \cdot g'(x)$$

Differentiate
outer function

Differentiate
inner function

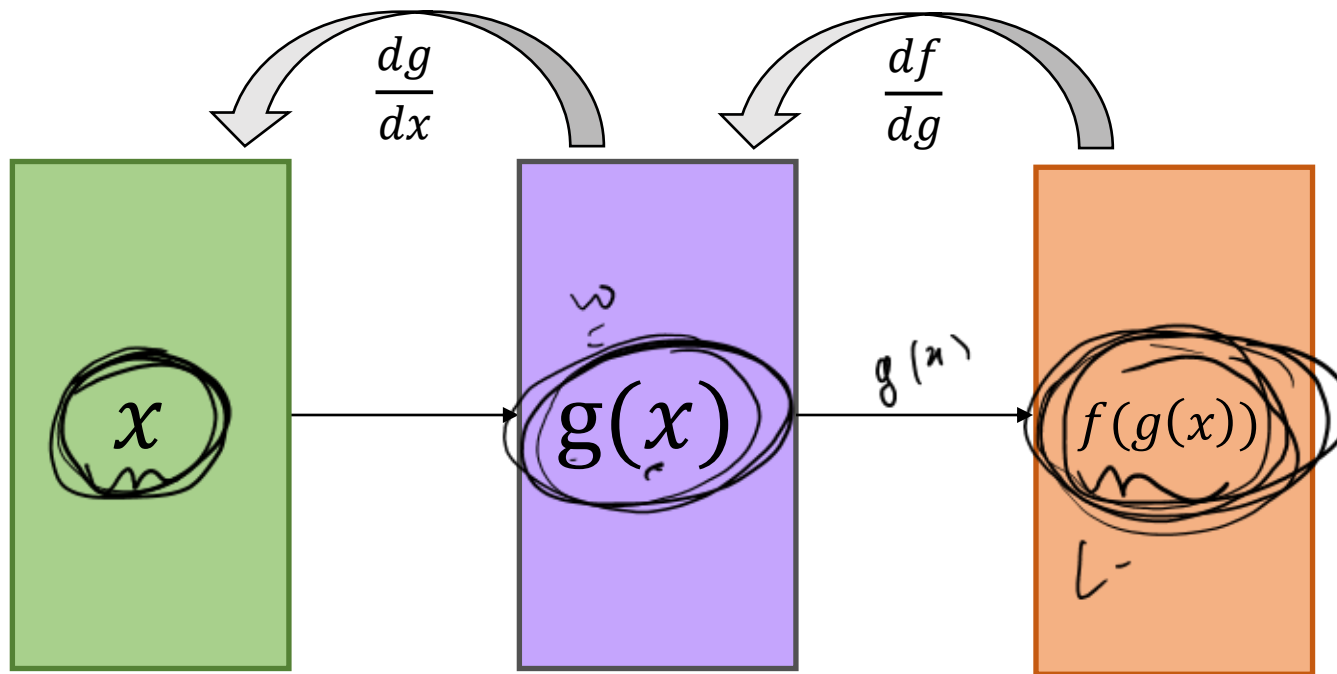
Applying Chain rule [Example]

$$\begin{array}{lcl} f(x) = x^2 & & g(x) = (2x^2 + 1) \rightarrow [4x] \\ \underbrace{\hspace{1cm}} & & \underbrace{\hspace{1cm}} \\ F(x) = f(g(x)) & & \underbrace{\hspace{1cm}} \\ F(x) = (2x^2 + 1)^2 & & \left\{ \begin{array}{l} z^2 \\ 2z \end{array} \right\} \end{array}$$

$$\begin{aligned} F'(x) &= 2z \cdot 4x \\ &= 2(2x^2 + 1) 4x \\ &= 16x^3 + 8x \\ &= 8x(2x^2 + 1) \end{aligned}$$

The Chain Rule (for Differentiation)

- Given arbitrary function: $f(g(x)) \Rightarrow \frac{df}{dx} = \frac{df}{dg} \frac{dg}{dx}$



Each layer computes the gradients with respect to its variables and passes the result backwards

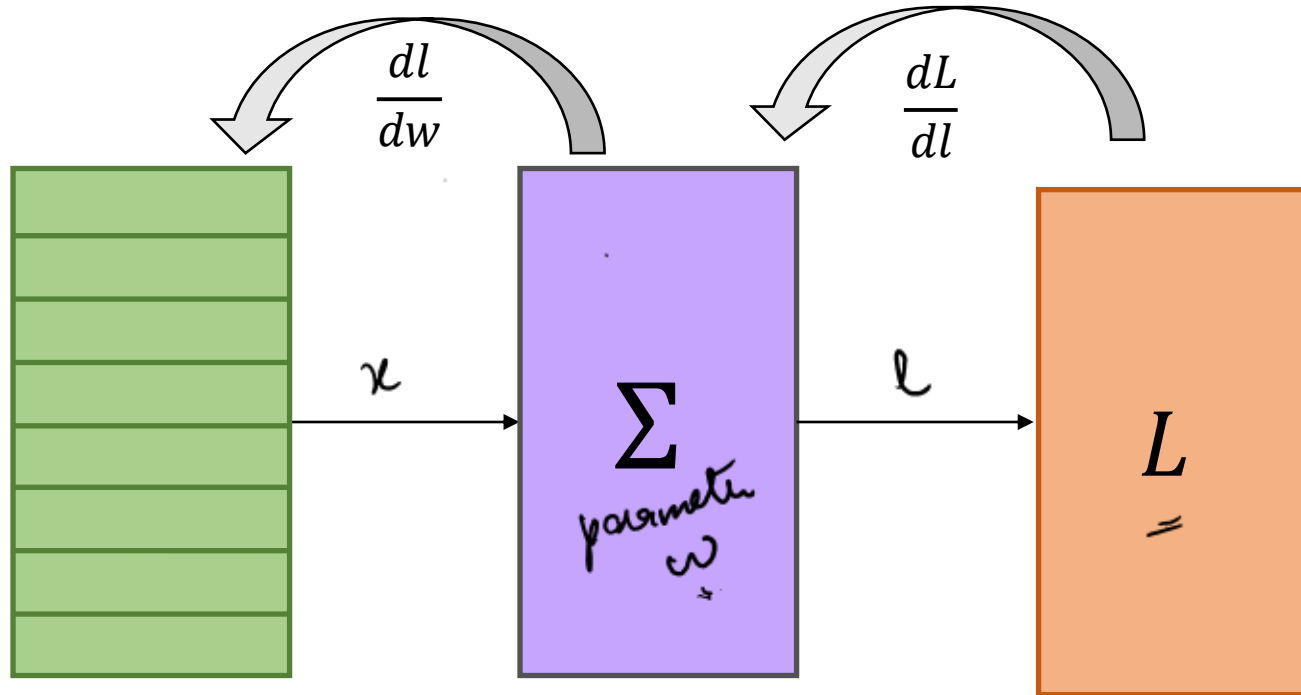
Backpropagation
(or backward pass)

$$\frac{\partial L}{\partial w}$$

The Chain Rule in Our Network

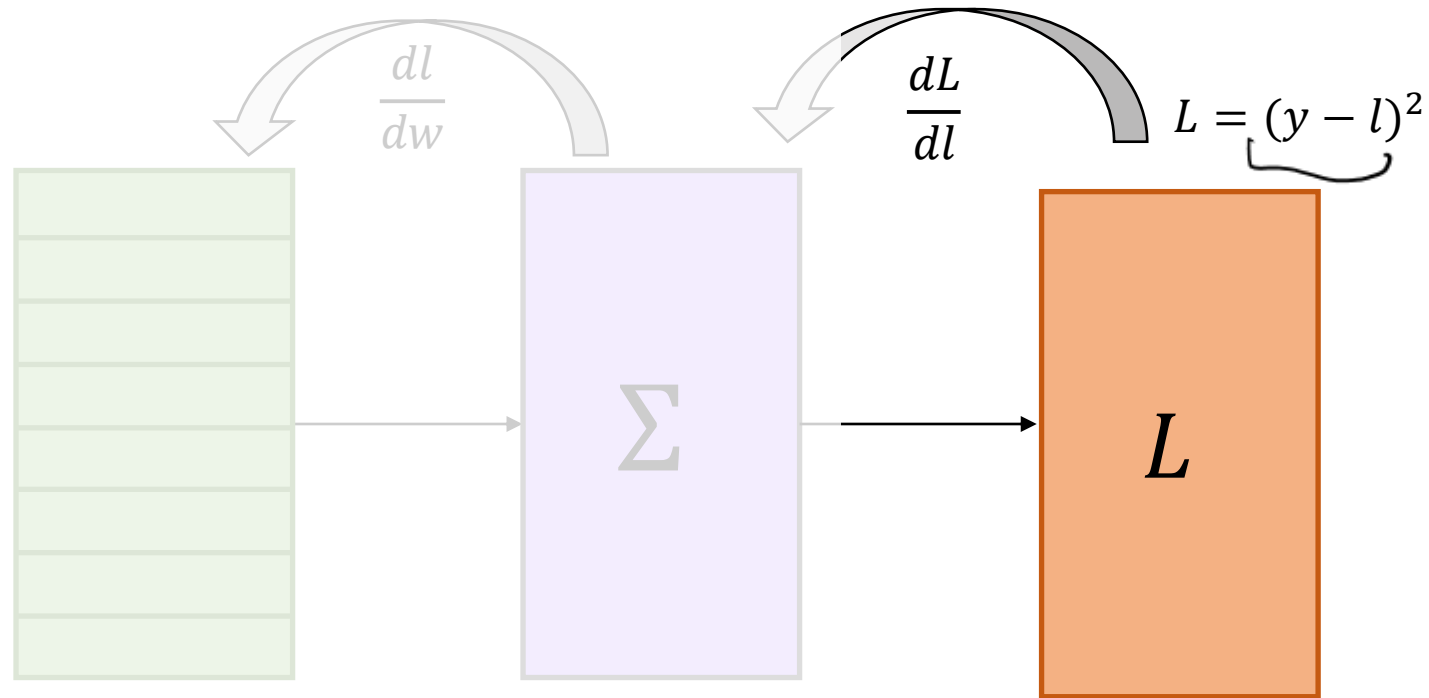
Remember – We calculate the gradient of L with respect to the parameters for learning them using gradient descent!

- Here's our function: $L(l(w)) \Rightarrow \underbrace{\frac{dL}{dw}} = \frac{dL}{dl} \cdot \frac{dl}{dw}$



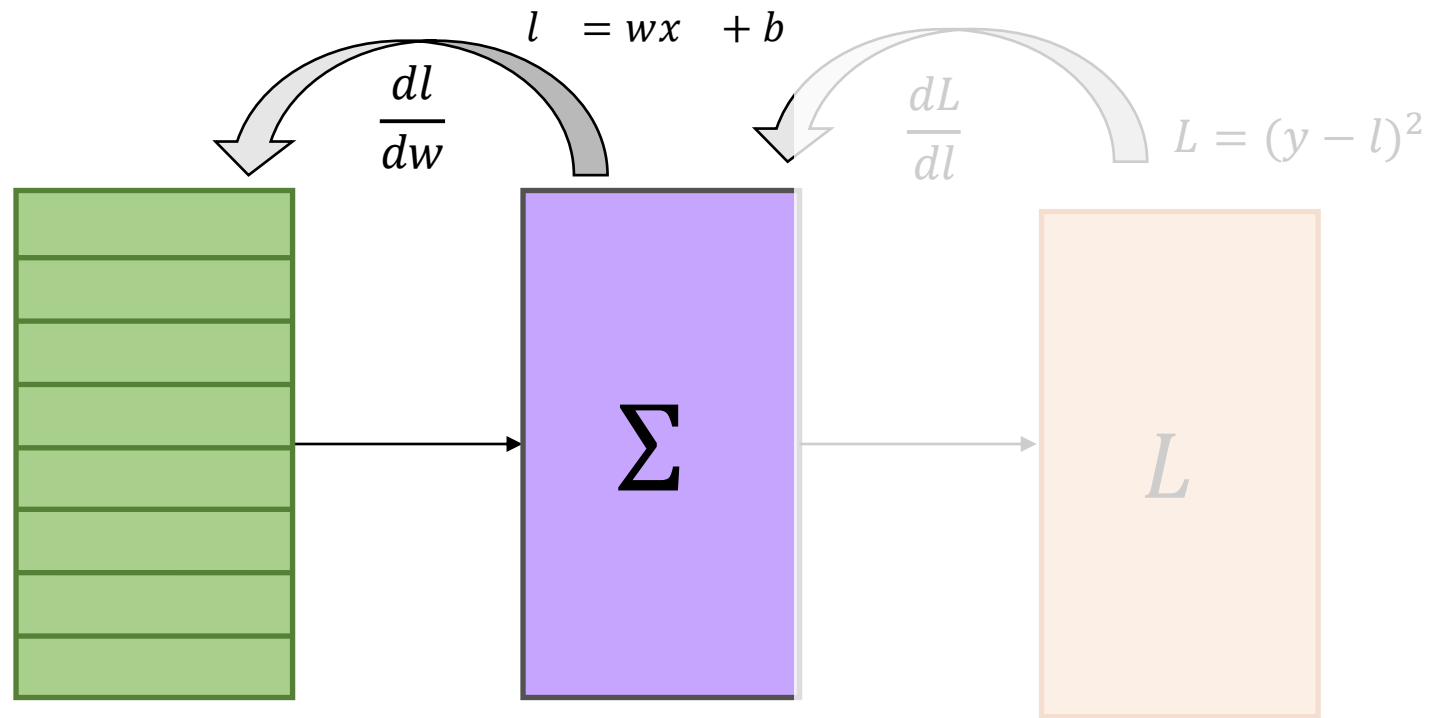
Derivative of loss layer

- $\frac{dL}{dl} = \frac{d(y-l)^2}{dl}$



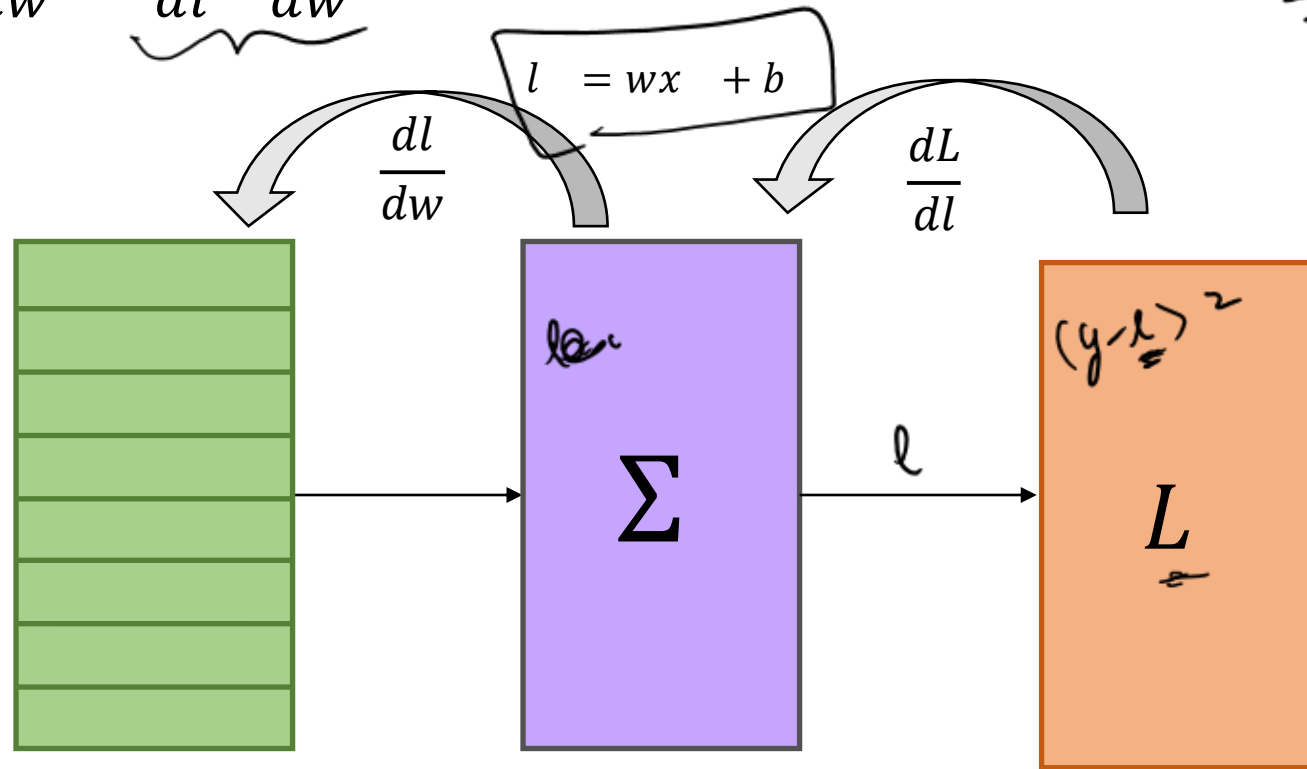
Derivative of linear layer

- $\frac{dl}{dw} = \frac{d(wx+b)}{dw}$



Putting it all together

- $\frac{dL}{dw} = \frac{dL}{dl} \cdot \frac{dl}{dw} =$

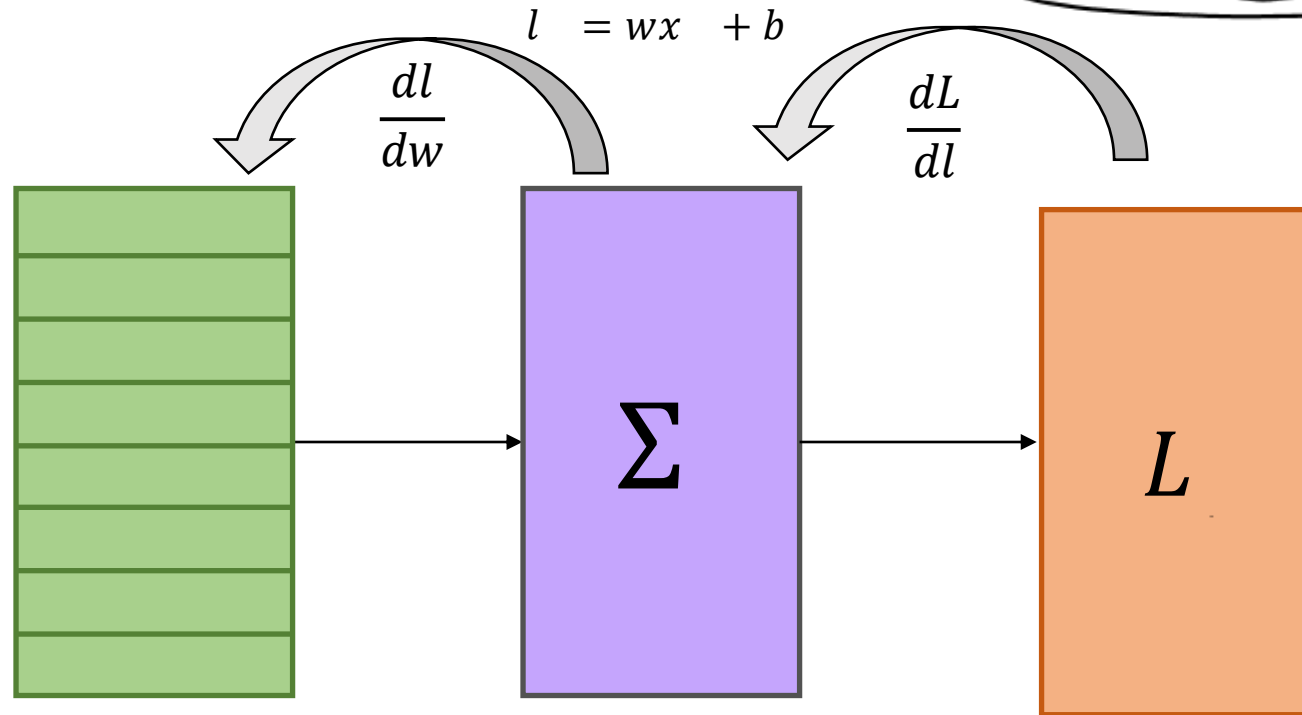


$$\frac{\partial L}{\partial l} = -2(y - l)$$
$$\frac{\partial L}{\partial w} \downarrow x$$

Putting it all together

Have we seen
this before?

$$\bullet \frac{dL}{dw} = \frac{dL}{dl} \cdot \frac{dl}{dw} = -2(y - l) \cdot x = \boxed{-2x(y - wx - b)} = 2x(wx + b - y)$$



Gradient Descent of MSE (1 sample)

$$\Delta w = -\alpha \cdot \frac{\partial L}{\partial w}$$

$$L = (y - \hat{y})^2$$

$$= (y - f(x))^2$$

$$= y^2 + f(x)^2 - 2yf(x)$$

$$= y^2 + (wx + b)^2 - 2y(wx + b)$$

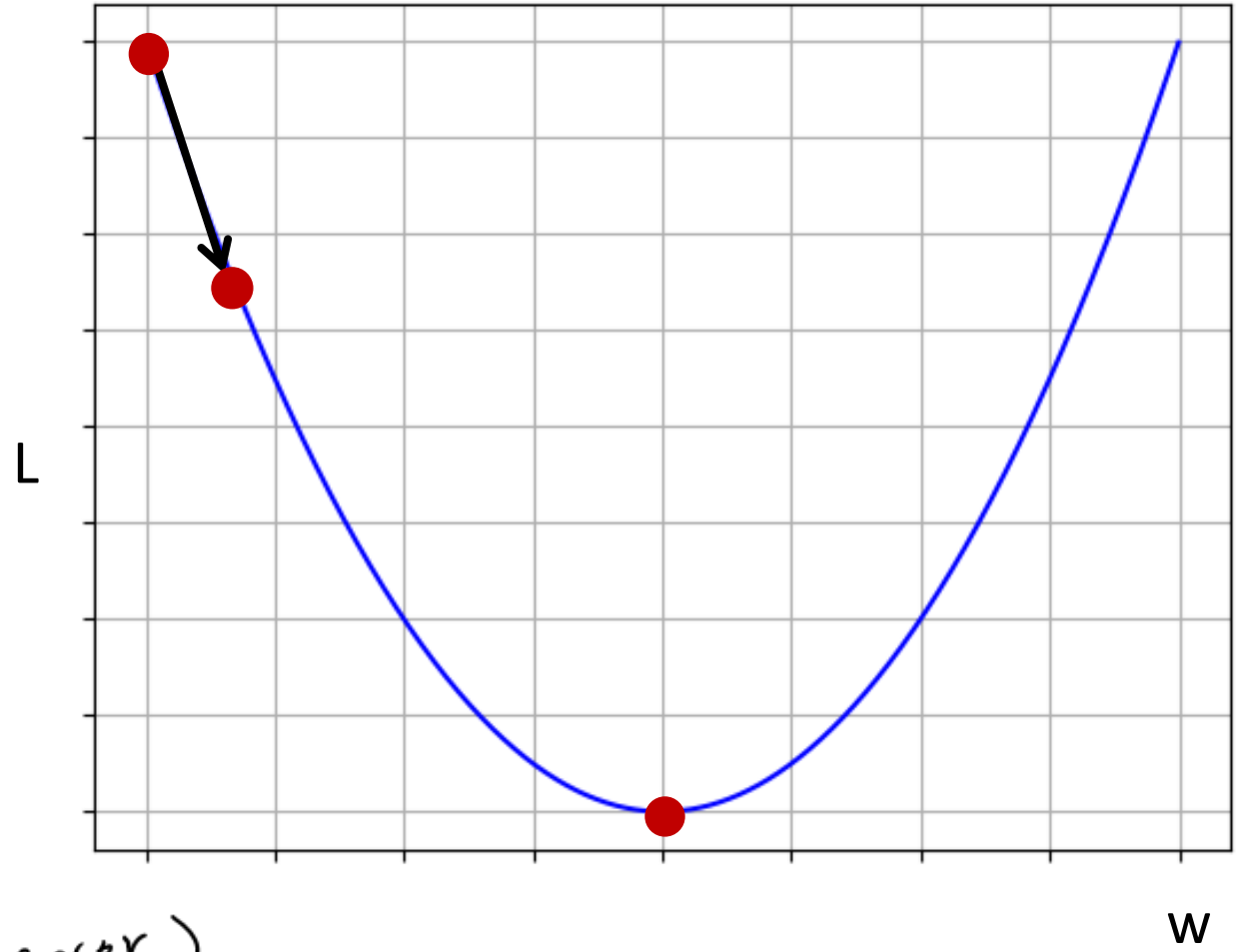
$$= y^2 + (wx + b)^2 - 2y(wx + b)$$

$$= y^2 + (wx + b)^2 - 2y(wx + b)$$

$$= y^2 + w^2x^2 + b^2 + 2wxb - 2ywx - 2yb$$

$$\frac{\partial L}{\partial w} = 2wx^2 + 2xb - 2yx$$

$$\frac{\partial L}{\partial w} = 2x(wx + b - y)$$



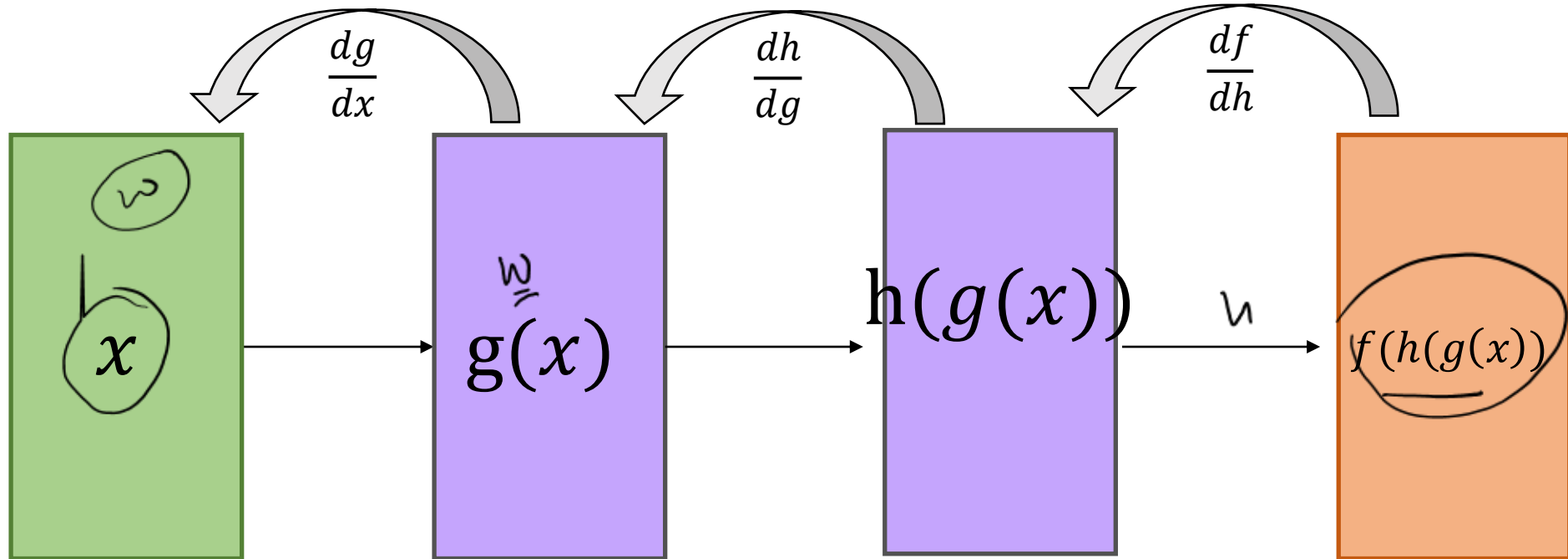
L (error)

Adding more layers!

- $f(h(g(x))) \Rightarrow \frac{df}{dx} = \frac{df}{dh} \cdot \frac{dh}{dg} \cdot \frac{dg}{dx}$

$$\frac{df}{dw} = \frac{df}{dh} \cdot \frac{dh}{dg} \cdot \frac{dg}{dw}$$

Can we add any function?



Any questions?



Few more important points: Backpropagation

- The process of calculating gradients of functions via chain rule in a neural network
- Is a part of and **NOT the whole learning algorithm**
- Can be calculated with respect to any variable of choice
- For learning in neural networks we calculate gradients with respect to the weights

Recap – Learn about fully connected neural networks

- (1) Multi-layered perceptron
- (2) Quick recap of loss functions and optimizer
- (3) Building our neural network

Wrap up



What was the clearest point today?

What was the muddiest point today?

